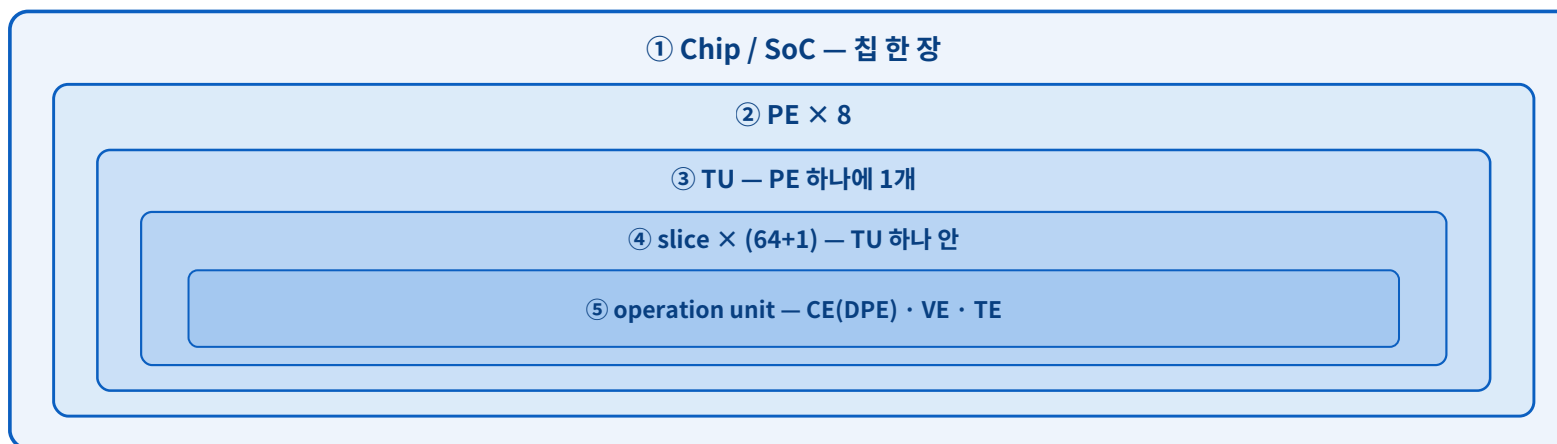


TCP 하드웨어 5계층과 소프트웨어 2용어

개념 하나에 그림 하나. 각 층이 무엇을 담고 있고, 몇 개씩 있으며, 무엇을 하는지를 그린다.

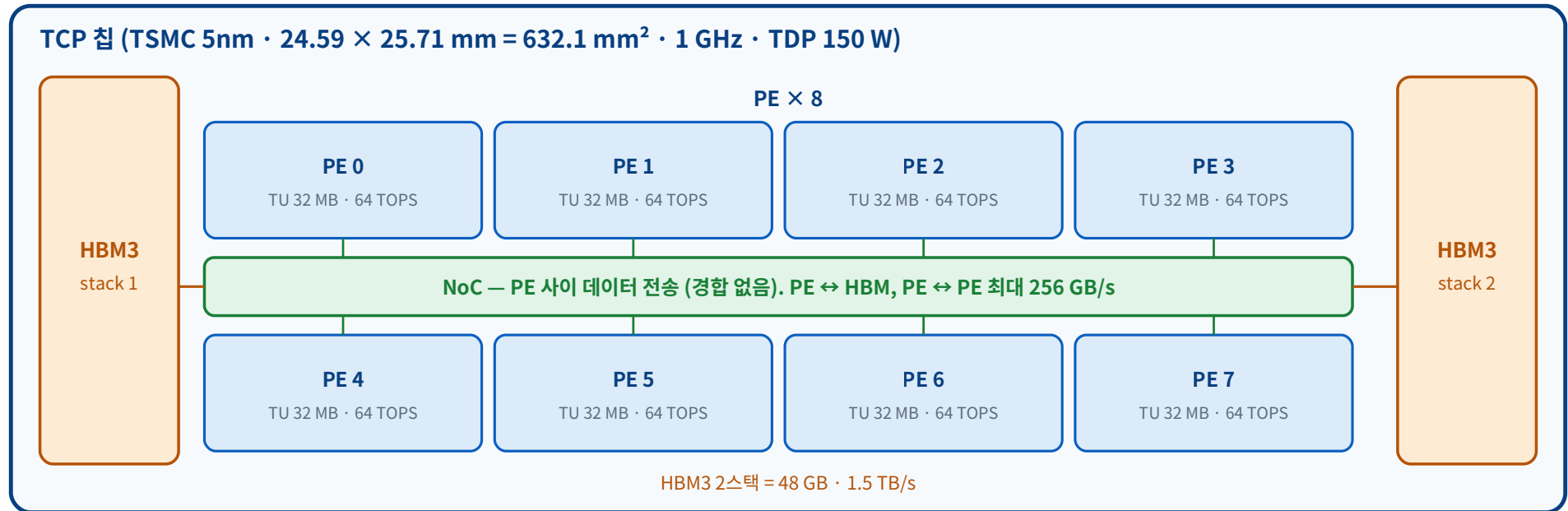
출처: TCP: A Tensor Contraction Processor for AI Workloads (ISCA 2024) 본문·그림



바깥이 안을 담는다. 오른쪽으로 갈수록 작아지고, 개수는 늘어난다.

① Chip / SoC — 칩 한 장

system-on-chip, 시스템 온 칩

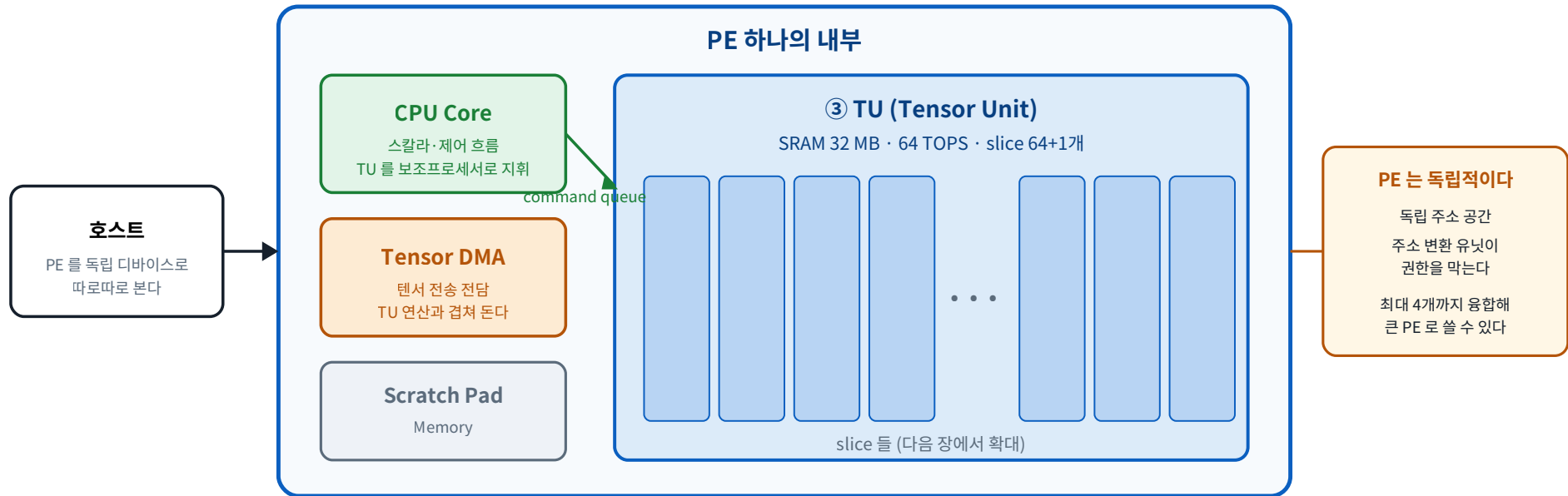


슬라이드 원문 — “1. Chip / SoC (system-on-chip, 시스템 온 칩) — TCP 칩 한 장”

- 칩 하나가 **PE 8개**와 **NoC**, **HBM3 2스택**, PCIe Gen5 × 16 을 담는다.
- 온칩 SRAM 합계 **256 MB** = PE당 32 MB × 8. 대역 384 TB/s.
- 연산 성능 512 TOPS (INT8) / 1024 TOPS (INT4) / 256 TFLOPS (BF16) / 512 TFLOPS (FP8).
- NoC 는 PE 사이 전송을 **경합 없이** 처리한다. PE 하나가 HBM 또는 다른 PE 로 최대 256 GB/s.
- PE 8개가 합쳐 HBM 대역 1.5 TB/s 를 다 쓸 수 있다.
- PCIe Gen5 로 칩 사이 P2P 통신도 지원해 여러 칩에 모델을 나눠 올릴 수 있다.

② PE — 처리 요소, 칩 하나에 8개

processing element · 호스트에서 독립 디바이스로 보일 수 있다

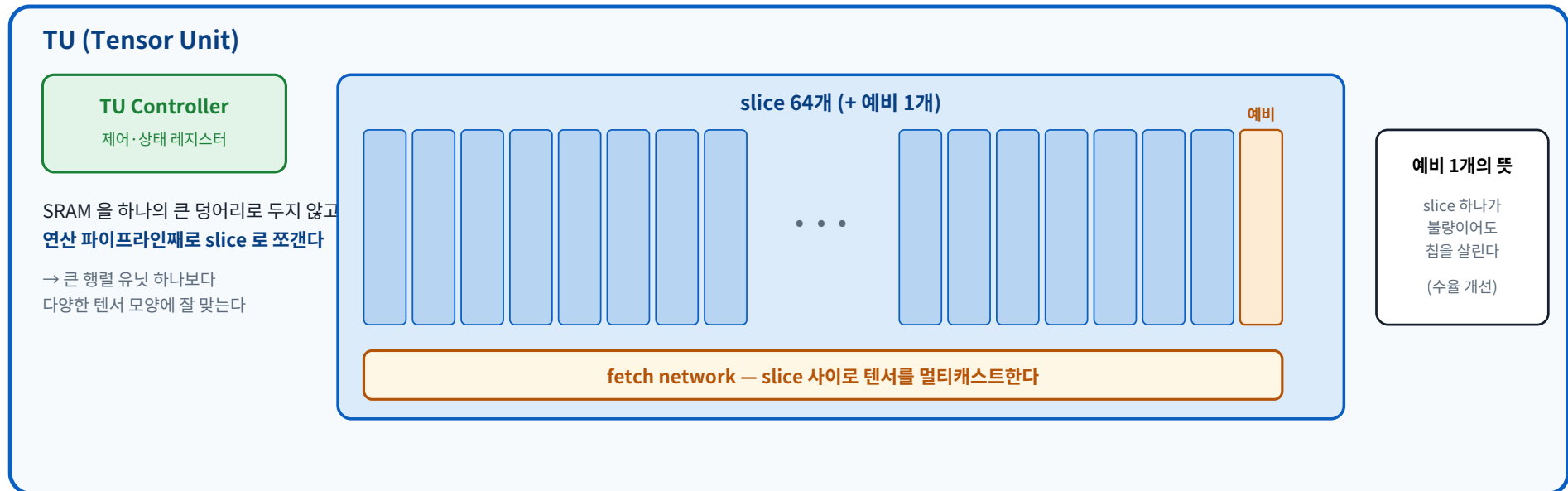


슬라이드 원문 — “2. PE (processing element, 처리 요소) — 칩 하나에 8개. 호스트에서 독립 디바이스로 보일 수 있다”

- PE = **CPU 코어 + TU + TDMA 엔진**(+ Scratch Pad Memory) 한 벌.
- CPU 코어가 스칼라 연산·제어 흐름을 맡고, TU 를 **보조프로세서처럼 커맨드 큐로** 지휘한다.
- 이전 텐서 연산이 도는 동안 다음 명령을 큐에 넣어 두므로 연산과 전송이 겹친다.
- PE 마다 **독립 주소 공간**을 갖고 독립적으로 동작한다. GPU 의 multi-instance 와 비슷하다.
- 최대 **4개까지 융합**해 하나의 큰 PE 로 쓸 수 있다. SR-IOV 로 가상머신에 나눠 줄 수도 있다.
- 그래서 “작게 여러 개”와 “크게 하나”를 상황에 맞춰 고를 수 있다.

③ TU — Tensor Unit, PE 안의 텐서 연산 본체

PE 하나에 1개 · SRAM 32 MB · 64 TOPS · slice 64+1개



슬라이드 원문 — “3. TU (Tensor Unit, 텐서 유닛) — PE 안에서 대규모 텐서 연산을 수행하는 단위”

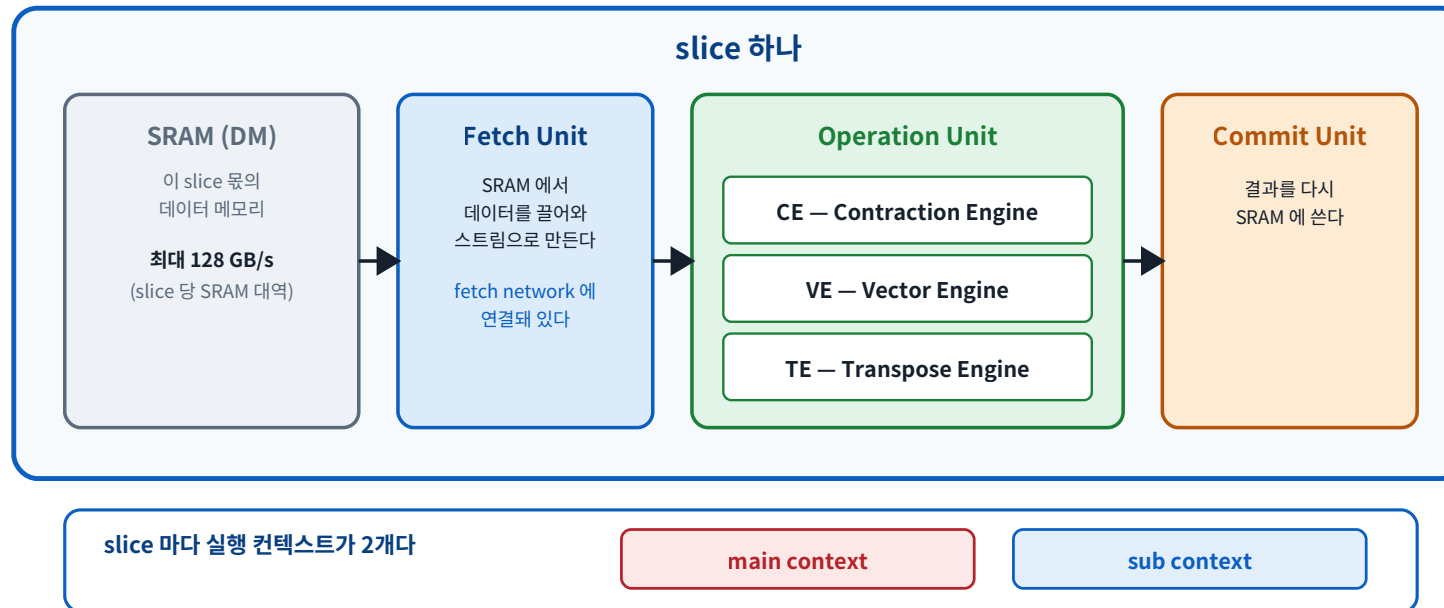
- TU 는 SRAM 에서 입력 텐서를 계속 끌어와 연산하고 결과를 다시 SRAM 에 쓴다.
- 큰 SRAM 하나 + 거대한 행렬곱 유닛이라는 통상적 설계를 **버렸다**. 대신 SRAM 을 포함한 파이프라인 전체를 slice 로 쪼갬다.

- 논문 본문: “**The TU includes 64+1 slices, with one reserved as spare to improve chip yields.**”
- Fig. 3 은 읽기 쉬우라고 slice 를 **8개만 그렸을 뿐**이다. 실제 수는 64+1 이다.

수 맞춰보기 — 칩당 slice = 8 PE × 64 = **512개**. VISA 문서가 말하는 “칩당 2 클러스터 × 256 slice = 512” 와 **수가 일치한다**. 다만 PE 와 클러스터의 정확한 대응 관계는 논문에 적혀 있지 않다.

④ slice — 기본 연산 단위

SRAM + fetch unit + operation unit + commit unit 을 한 벌씩 가진다



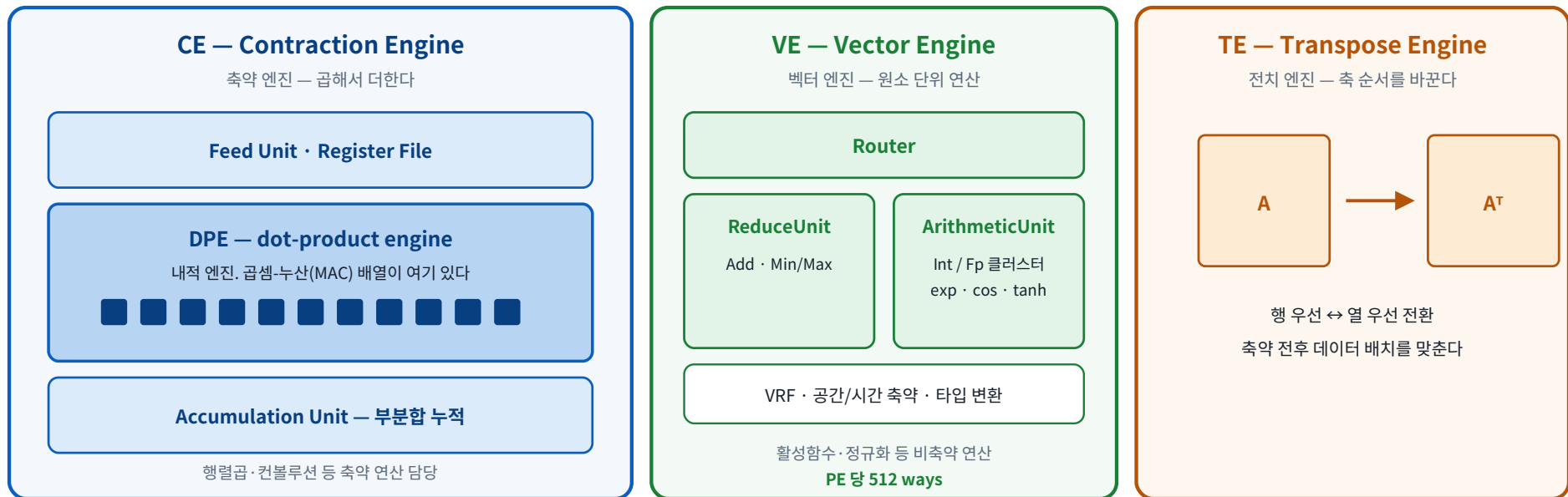
슬라이드 원문 — “4. slice (슬라이스) — TU 를 쪼갬 기본 연산 단위. SRAM + fetch unit + operation unit + commit unit 을 한 벌씩 가진다”

- 논문 본문: “each slice comprises of SRAM, fetch, operation, and commit units.”
- slice 는 제어 레지스터로 설정되며 독립적으로 (부분)텐서를 처리할 수 있다.

- fetch network 가 slice 들을 이어 여러 slice 로 텐서를 멀티캐스트한다. N개 slice 에서 읽어 퍼뜨리면 큰 SRAM 의 N개 뱅크를 읽는 것과 같아진다.
- slice 마다 실행 컨텍스트가 둘(main / sub)이라 연산과 준비를 겹칠 수 있다(Fig. 4).

⑤ operation unit — slice 안의 연산기 3종

CE (그 안에 DPE) · VE · TE

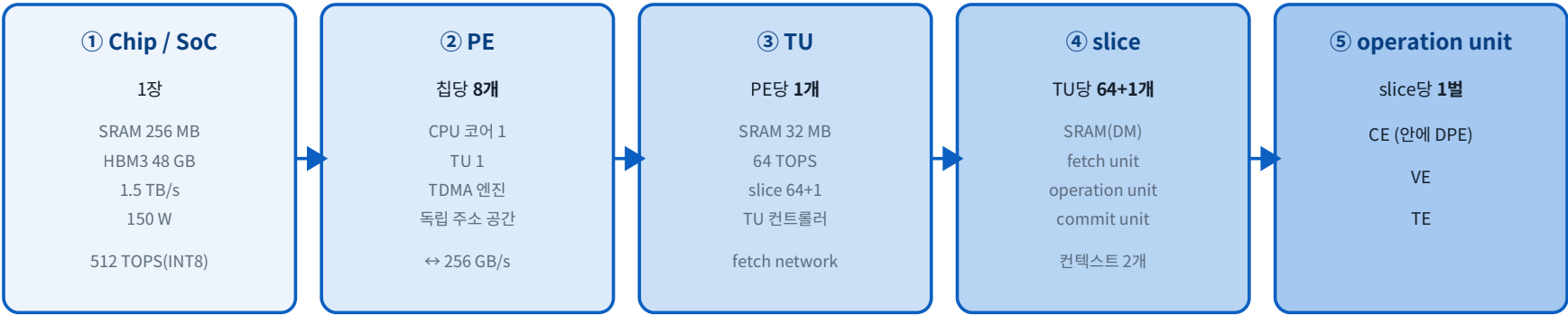


슬라이드 원문 — “5. operation unit (연산 유닛) — slice 안의 CE / VE / TE · CE = Contraction Engine (축약 엔진), 그 안에 DPE = dot-product engine (내적 엔진) · VE = Vector Engine (벡터 엔진), TE = Transpose Engine (전치 엔진)”

논문 본문: “the operation units, each of which consists of contraction (CE), vector (VE), and transpose (TE) engines.” — 즉 operation unit 은 이 셋을 묶어 부르는 이름이지 별개의 부품이 아니다.

5계층 한 장 정리 — 개수와 용량

바깥에서 안으로 · 논문에 적힌 값만



층	개수	담는 것	논문 근거
① Chip / SoC	—	PE 8 · NoC · HBM3 2스택 · PCIe Gen5	III 절, TABLE I, Fig. 2
② PE	칩당 8	CPU 코어 + TU + TDMA(+ Scratch Pad)	III-A, Fig. 3
③ TU	PE당 1	SRAM 32 MB · 64 TOPS · slice 64+1	III-B (“The TU includes 64+1 slices...”)
④ slice	TU당 64+1	SRAM + fetch + operation + commit	III-B, Fig. 4
⑤ operation unit	slice당 1벌	CE(DPE) · VE · TE	III-B, IV 절

주의 — Fig. 3 캡션은 “...with eight slices” 라서 그림만 보면 slice 가 8개로 보인다. 본문이 말하는 실제 수는 **64+1** 이며, 그림은 가독성을 위해 줄여 그린 것이다.

소프트웨어 용어 ① — lowered shape (내려앉힌 형태)

텐서를 어느 축으로 어떻게 잘라 어느 slice 에 올릴지를 적은 것

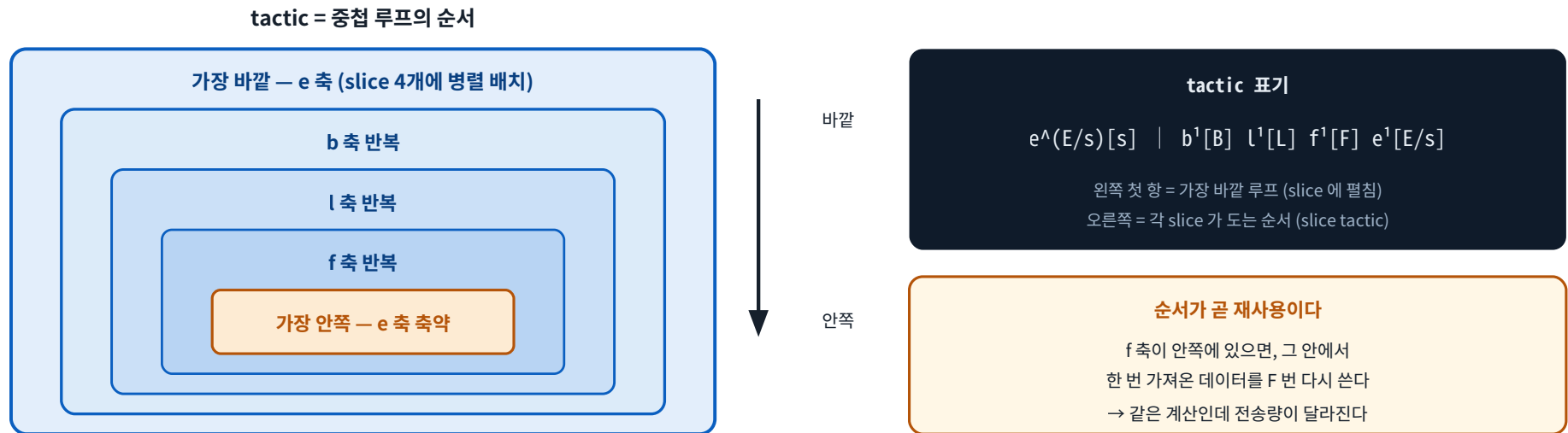


슬라이드 원문 — “lowered shape (내려앉힌 형태) — 텐서를 어느 축으로 어떻게 잘라 어느 slice 에 올릴지를 적은 것”

- 논문 용어로는 “**inter-slice partitioning shape** | **intra-slice shape**”. 세로줄을 사이에 두고 왼쪽이 slice 사이 분할, 오른쪽이 slice 안 모양이다.
- 같은 텐서라도 **자르는 방법이 여러 가지**다. Fig. 1 은 같은 $b \times l \times e$ 텐서의 세 가지 lowered shape 을 나란히 보여준다.
- 어떻게 자르냐에 따라 재사용 가능한 데이터와 필요한 전송량이 달라진다. 그래서 **성능이 여기서 갈린다**.
- 컴파일러가 후보들을 탐색해 요구 성능·전력을 만족하는 것을 고른다.

소프트웨어 용어 ② — tactic (택틱, 전술)

축을 바깥에서 안쪽으로 어떤 순서로 도는지, 즉 루프 순서를 적은 것



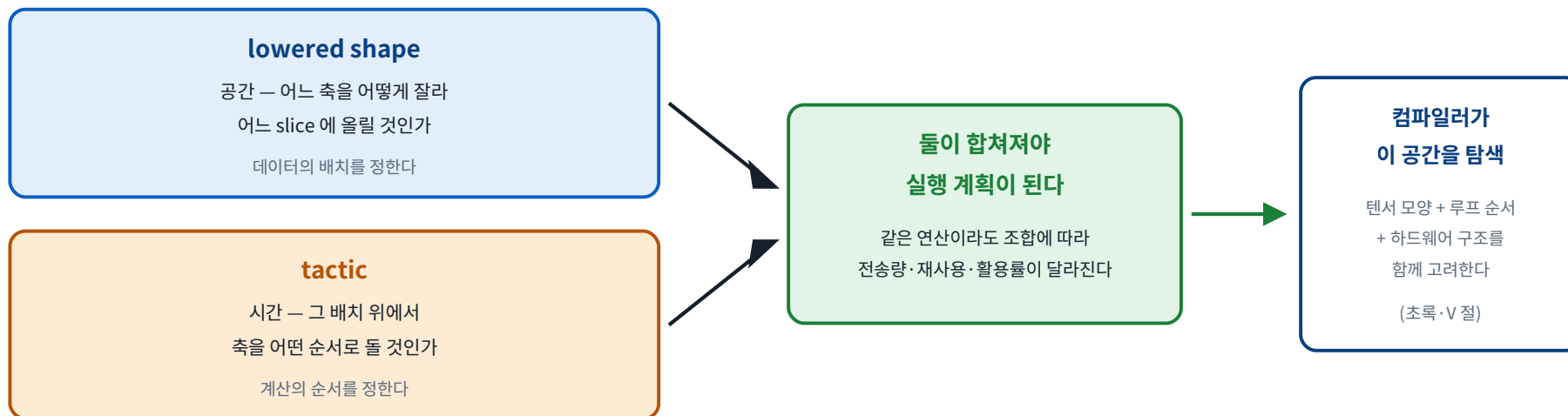
슬라이드 원문 — “tactic (택틱, 전술) — 축을 바깥에서 안쪽으로 어떤 순서로 도는지, 즉 루프 순서를 적은 것”

- 논문 정의: “A tactic for contraction describes the order of computing axes, from the outermost to the innermost.”
- 가장 바깥 항은 slice 에 공간적으로 펼치는 축이고, 나머지는 각 slice 안에서 시간적으로 도는 순서다.

- lowered shape 이 같아도 tactic 이 다르면 성능이 달라진다. Fig. 8 이 같은 축약 `ble,ef` 에 대해 세 가지 조합을 비교한다.
- 컴파일러가 이 설계 공간을 탐색한다. 그래서 tactic 은 사람이 고르거나 컴파일러가 고르는 선택지다.

lowered shape 과 tactic 은 짝이다

하나는 어디에 놓을지, 다른 하나는 어떤 순서로 돌지를 정한다



슬라이드 원문 — “서론에는 이 용어들이 정의 없이 먼저 등장한다. 정의는 II절에 있다”

- 서론(I 절)은 lowered shape · tactic 을 설명 없이 쓴다. 처음 읽으면 여기서 막힌다.
- 정의는 II. PRELIMINARIES 에 있다 — II-A 가 lowered shape, II-B 가 tactic 이다.

- 그래서 이 논문은 II 절을 먼저 읽고 I 절로 돌아오는 편이 빠르다.
- V 절(프로그래밍 인터페이스)과 VI 절(LLaMA-2 사례)은 이 두 용어 위에서만 읽힌다.